Crucible — Forge Architecture

> *Agentic scenario intake pipeline. Input: free-text scenario description from a consultant. Output: a fully configured `SimSpec` ready to hand to `SimRunner`.*
>
> ***The promise:*** *"Describe a scenario on Monday. By Tuesday, a calibrated, running simulation is delivering insights."*

Overview

```
Consultant free text
        │
        ▼
┌──────────────────────────────────────────────────────────────┐
│  ForgeSession (conversation state object)                    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  ScopingAgent  (Claude API tool-use agent)            │    │
│  │                                                       │    │
│  │  INTAKE ──► PARALLEL_RESEARCH ──► DYNAMIC_INTERVIEW   │    │
│  │               │                      │                │    │
│  │               │   (async, fan-out)    │                │    │
│  │           ┌───┴───┐             gap detection         │    │
│  │           │Research │             question gen         │    │
│  │           │Pipeline │             user answers         │    │
│  │           └───┬───┘                  │                │    │
│  │               │                      ▼                │    │
│  │               └──────────► THEORY_MAPPING             │    │
│  │                                  │                    │    │
│  │                            TheoryMapper               │    │
│  │                                  │                    │    │
│  │                                  ▼                    │    │
│  │                          VALIDATION ──► COMPLETE      │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│   SimSpec (grows incrementally across all states)            │
└──────────────────────────────────────────────────────────────┘
        │
        ▼ (on COMPLETE)
┌──────────────┐
│  SimRunner   │  setup() → run() → snapshots → metric_history
└──────────────┘
```

---

1. `forge/session.py` — ForgeSession

The central conversation state object. One per intake conversation. Serializable to JSON for persistence.

```python
from __future__ import annotations

import time
import uuid
from dataclasses import dataclass, field
from enum import Enum
from typing import Any

from core.spec import SimSpec


# ── State machine ──────────────────────────────────────────────────────

class ForgeState(str, Enum):
    INTAKE            = "intake"            # parsing free text, no research yet
    PARALLEL_RESEARCH = "parallel_research" # background research fired, waiting
    DYNAMIC_INTERVIEW = "dynamic_interview" # asking gap-filling questions
    THEORY_MAPPING    = "theory_mapping"    # TheoryMapper selecting + parameterizing
    VALIDATION        = "validation"        # final SimSpec validation pass
    COMPLETE          = "complete"          # SimSpec handed to SimRunner


# ── Message types ──────────────────────────────────────────────────────

class MessageRole(str, Enum):
    USER      = "user"
    ASSISTANT = "assistant"
    TOOL      = "tool"      # tool result injected into context
    SYSTEM    = "system"    # internal state transitions, not shown to user


@dataclass
class ForgeMessage:
    role:        MessageRole
    content:     str
    tool_name:   str | None  = None    # set when role == TOOL
    tool_use_id: str | None  = None    # Claude API tool_use_id for correlation
    timestamp:   float       = field(default_factory=time.time)
    metadata:    dict[str, Any] = field(default_factory=dict)


# ── Gap tracking ───────────────────────────────────────────────────────

@dataclass
class SpecGap:
    """
    Represents a missing or under-specified part of the SimSpec.
    Drives question ordering: high priority gaps are asked first.
    """
    gap_id:          str   = field(default_factory=lambda: str(uuid.uuid4()))
    field_path:      str   = ""     # e.g. "actors", "initial_environment", "theories"
    description:     str   = ""     # human-readable: what specifically is missing
    priority:        float = 0.5    # 0–1. 1.0 = must ask. 0.0 = nice to have.
    filled:          bool  = False
    filled_by:       str | None = None  # "research" | "user" | "inference"
    question_asked:  str | None = None
    answer_received: str | None = None


# ── Research outputs ───────────────────────────────────────────────────

@dataclass
class ResearchResult:
    source:            str            # "arxiv" | "ssrn" | "fred" | "world_bank" | "news"
    id:                str            # source-native identifier
    title:             str
```

```python
    summary:            str            # abstract or description
    theory_suggestions: list[str]      # LLM-extracted: e.g. ["richardson_arms_race"]
    parameter_hints:    dict[str, float]  # LLM-extracted: e.g. {"richardson_arms_race__a": 0.25}
    relevance_score:    float          # 0–1, LLM-assessed
    url:                str
    raw:                dict[str, Any] = field(default_factory=dict)


@dataclass
class DataPoint:
    source:    str            # "fred" | "world_bank"
    series_id: str
    label:     str
    value:     float
    date:      str            # ISO 8601
    unit:      str = ""
    url:       str = ""


@dataclass
class NewsItem:
    source:          str
    headline:        str
    summary:         str
    url:             str
    published:       str       # ISO 8601
    relevance_score: float = 0.0


@dataclass
class ResearchContext:
    session_id:          str
    results:             list[ResearchResult] = field(default_factory=list)
    data_points:         list[DataPoint]      = field(default_factory=list)
    news_items:          list[NewsItem]       = field(default_factory=list)
    # consolidated across all results after extraction pass
    theory_candidates:   list[str]            = field(default_factory=list)
    parameter_estimates: dict[str, float]     = field(default_factory=dict)
    # which env keys research has answered (so we don't ask the user)
    env_keys_calibrated: set[str]             = field(default_factory=set)
    research_complete:   bool                 = False


# ── ForgeSession ──────────────────────────────────────────────────────

@dataclass
class ForgeSession:
    session_id:           str             = field(default_factory=lambda: str(uuid.uuid4()))
    simspec:              SimSpec | None  = None
    research_context:     ResearchContext = field(
        default_factory=lambda: ResearchContext(session_id="")
    )
    conversation_history: list[ForgeMessage] = field(default_factory=list)
    state:                ForgeState      = ForgeState.INTAKE
    gaps:                 list[SpecGap]   = field(default_factory=list)
    turn_count:           int             = 0
    created_at:           float           = field(default_factory=time.time)
    completed_at:         float | None    = None
    intake_text:          str             = ""
    domain:               str             = ""   # "geopolitics" | "market" | "macro" | "org"

    def open_gaps(self) -> list[SpecGap]:
        """Return unfilled gaps ordered by priority descending."""
        return sorted(
            [g for g in self.gaps if not g.filled],
            key=lambda g: g.priority,
            reverse=True,
        )

    def mark_complete(self) -> None:
```

```
        self.state = ForgeState.COMPLETE
        self.completed_at = time.time()
```

---

2. State Machine — `forge/scoping_agent.py`

The `ScopingAgent` is a Claude API tool-use agent. It drives all state transitions.

State Transitions

```
┌──────────────────────────────┐
│          INTAKE              │
│  parse free text via Claude API  │
│  extract: domain, actors,    │
│  timeframe (rough)           │
│  build skeleton SimSpec      │
└──────────────┬───────────────┘
               │ (immediately, no user prompt)
               ▼
┌──────────────────────────────┐
│      PARALLEL_RESEARCH       │
│  asyncio.gather:             │
│    search_arxiv()            │
│    search_ssrn()             │
│    search_fred()             │
│    search_world_bank()       │
│    search_news()             │
│  → extraction pass (Claude API) │
│  → populate ResearchContext  │
└──────────────┬───────────────┘
               │ research_context.research_complete = True
               ▼
┌──────────────────────────────┐
│      DYNAMIC_INTERVIEW       │
│  detect_gaps(simspec)        │
│  subtract: research-filled gaps │
│  generate ≤5 questions       │
│  ask_user() → user answers   │
│  update_simspec() per answer │
│  loop until gaps closed or   │
│  turn_count ≥ MAX_TURNS (5)  │
└──────────────┬───────────────┘
               │ open_gaps() empty OR turn_count ≥ MAX_TURNS
               ▼
┌──────────────────────────────┐
│       THEORY_MAPPING         │
│  select_theories()           │
│  TheoryMapper.map()          │
│  update simspec.theories     │
└──────────────┬───────────────┘
               │
```

```
                    ▼
        ┌──────────────────────────────┐
        │          VALIDATION          │
        │  finalize()                  │
        │  SimSpec.model_validate()    │
        │  if fail → re-open gaps      │
        │            back to INTERVIEW │
        └──────────────────────────────┘
                    │
                    │ validation passes
                    ▼
                 COMPLETE
```

`ScopingAgent` class

```python
import asyncio
from anthropic import Anthropic

from core.spec import SimSpec
from forge.session import (
    ForgeSession, ForgeState, ForgeMessage, MessageRole, SpecGap,
)
from forge.researchers import (
    ArxivResearcher, SsrnResearcher, FredResearcher,
    WorldBankResearcher, NewsResearcher,
)
from forge.theory_mapper import TheoryMapper
from forge.spec_builder import SpecBuilder
from forge.gap_detector import detect_gaps, _merge_gaps


MAX_TURNS = 5   # max interview turns before auto-completing the spec


class ScopingAgent:
    """
    Claude API tool-use agent that drives a ForgeSession from INTAKE to COMPLETE.

    Usage:
        session = ScopingAgent.create_session(intake_text)
        reply   = await agent.turn(session, user_message=None)   # first turn
        reply   = await agent.turn(session, user_message="...")  # subsequent turns
    """

    SYSTEM_PROMPT = """
You are the Forge intake agent for the Crucible simulation platform.
Your job: guide a consultant through describing a scenario so you can
build a complete SimSpec — a fully configured simulation specification.

Use research tools BEFORE asking any questions. Your questions must be
informed by research. Never ask for something research already answered.

Use consulting language. Never expose model parameters directly.
("How do you expect actor X to respond to economic pressure?"
 not "What should I set richardson_arms_race__a to?")

Complete the spec in ≤ 5 questions. If a gap cannot be filled with
certainty, use a reasonable default informed by research and flag it
in the SimSpec metadata.
"""

    def __init__(self, client: Anthropic | None = None) -> None:
        self.client = client or Anthropic()
        self.theory_mapper = TheoryMapper()
```

```python
        self.spec_builder = SpecBuilder(self.client)

    @staticmethod
    def create_session(intake_text: str) -> ForgeSession:
        session = ForgeSession(intake_text=intake_text)
        session.research_context.session_id = session.session_id
        return session

    async def turn(
        self,
        session: ForgeSession,
        user_message: str | None = None,
    ) -> str:
        """
        Drive the session forward by one turn.
        Returns the next message to show the consultant.
        """
        if user_message:
            session.conversation_history.append(
                ForgeMessage(role=MessageRole.USER, content=user_message)
            )
            session.turn_count += 1

        if session.state == ForgeState.INTAKE:
            return await self._run_intake(session)
        if session.state == ForgeState.DYNAMIC_INTERVIEW:
            return await self._run_interview_turn(session, user_message)
        if session.state == ForgeState.THEORY_MAPPING:
            return await self._run_theory_mapping(session)
        if session.state == ForgeState.VALIDATION:
            return await self._run_validation(session)

        return "Simulation specification complete. Handing off to SimRunner."

    # ── State handlers ──────────────────────────────────────────────────

    async def _run_intake(self, session: ForgeSession) -> str:
        skeleton = self.spec_builder.parse_intake(session.intake_text)
        session.simspec = skeleton
        session.domain = skeleton.domain
        session.state = ForgeState.PARALLEL_RESEARCH

        await self._run_research(session)

        self.spec_builder.apply_research_hints(session.simspec, session.research_context)
        session.gaps = detect_gaps(session.simspec)
        self._mark_research_filled_gaps(session)

        session.state = ForgeState.DYNAMIC_INTERVIEW
        return await self._run_interview_turn(session, user_message=None)

    async def _run_research(self, session: ForgeSession) -> None:
        """Fan-out research across all adapters in parallel."""
        domain_queries = _build_domain_queries(session.simspec, session.intake_text)

        results = await asyncio.gather(
            ArxivResearcher().search(domain_queries["academic"], max_results=5),
            SsrnResearcher().search(domain_queries["academic"], max_results=5),
            FredResearcher().fetch(domain_queries.get("fred_series", [])),
            WorldBankResearcher().fetch(
                domain_queries.get("wb_indicators", []),
                domain_queries.get("wb_countries", []),
            ),
            NewsResearcher().search(domain_queries["news"], days_back=90),
            return_exceptions=True,  # research failure is non-fatal
        )

        ctx = session.research_context
        for result in results:
            if isinstance(result, Exception):
```

```python
            continue  # log but continue
        if isinstance(result, list):
            for item in result:
                _classify_result(item, ctx)

    await self._extraction_pass(ctx)
    ctx.research_complete = True

async def _extraction_pass(self, ctx) -> None:
    """
    Single Claude API call that reads all research summaries and extracts:
    - theory_suggestions and parameter_hints per result
    - consolidated theory_candidates and parameter_estimates on ctx

    Prompt (system): extract JSON {"results": [{"id": ..., "theory_suggestions": [...],
                      "parameter_hints": {...}}]} from research summaries.
    Updates ctx in-place.
    """
    ...

async def _run_interview_turn(
    self,
    session: ForgeSession,
    user_message: str | None,
) -> str:
    if user_message:
        self.spec_builder.apply_user_answer(
            session.simspec, session.gaps, user_message
        )
        new_gaps = detect_gaps(session.simspec)
        _merge_gaps(session.gaps, new_gaps)
        self._mark_research_filled_gaps(session)

    open_gaps = session.open_gaps()
    if not open_gaps or session.turn_count >= MAX_TURNS:
        self._auto_fill_gaps(session)
        session.state = ForgeState.THEORY_MAPPING
        return await self._run_theory_mapping(session)

    next_gap = open_gaps[0]
    question = await self._generate_question(next_gap, session)
    next_gap.question_asked = question
    session.conversation_history.append(
        ForgeMessage(role=MessageRole.ASSISTANT, content=question)
    )
    return question

async def _run_theory_mapping(self, session: ForgeSession) -> str:
    theories = self.theory_mapper.map(session.simspec, session.research_context)
    session.simspec = session.simspec.model_copy(
        update={"theories": [t.model_dump() for t in theories]}
    )
    session.state = ForgeState.VALIDATION
    return await self._run_validation(session)

async def _run_validation(self, session: ForgeSession) -> str:
    from pydantic import ValidationError
    try:
        SimSpec.model_validate(session.simspec.model_dump())
        session.mark_complete()
        return (
            f"Your simulation '{session.simspec.name}' is configured and ready to run. "
            f"Actors: {len(session.simspec.actors)}. "
            f"Theories: {', '.join(t.theory_id for t in session.simspec.theories)}."
        )
    except ValidationError as e:
        for error in e.errors():
            gap = SpecGap(
                field_path=".".join(str(x) for x in error["loc"]),
                description=error["msg"],
```

```
                priority=1.0,
            )
            session.gaps.append(gap)
        session.state = ForgeState.DYNAMIC_INTERVIEW
        return await self._run_interview_turn(session, user_message=None)

    # ── Helpers ──────────────────────────────────────────────────────────

    def _mark_research_filled_gaps(self, session: ForgeSession) -> None:
        calibrated = session.research_context.env_keys_calibrated
        for gap in session.gaps:
            if not gap.filled and gap.field_path in calibrated:
                gap.filled = True
                gap.filled_by = "research"

    def _auto_fill_gaps(self, session: ForgeSession) -> None:
        for gap in session.open_gaps():
            self.spec_builder.infer_gap(session.simspec, gap, session.research_context)
            gap.filled = True
            gap.filled_by = "inference"

    async def _generate_question(self, gap: SpecGap, session: ForgeSession) -> str:
        """Claude API call using QUESTION_GENERATION_PROMPT template."""
        ...  # see Section 8
```

---

3. Tool Schema

Tools exposed to the Claude API. Each maps to a method on `ScopingAgent`.

```
FORGE_TOOLS = [
    {
        "name": "search_arxiv",
        "description": (
            "Search arXiv preprints for academic research relevant to the scenario. "
            "Use for theoretical frameworks, empirical studies, domain models."
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "query":       {"type": "string"},
                "max_results": {"type": "integer", "default": 5, "maximum": 10},
            },
            "required": ["query"],
        },
    },
    {
        "name": "search_ssrn",
        "description": "Search SSRN for working papers in economics, finance, law, and social science.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query":       {"type": "string"},
                "max_results": {"type": "integer", "default": 5, "maximum": 10},
            },
            "required": ["query"],
        },
    },
    {
        "name": "search_fred",
        "description": (
            "Fetch time series data from FRED. "
            "Use for macroeconomic calibration: GDP, inflation, trade flows, interest rates."
```

```
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "series_ids": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "FRED series IDs, e.g. ['GDP', 'CPIAUCSL', 'UNRATE']",
                },
            },
            "required": ["series_ids"],
        },
    },
    {
        "name": "search_world_bank",
        "description": (
            "Fetch World Bank development indicators for specified countries. "
            "Use for cross-country macro calibration: GDP per capita, trade %, military spend %."
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "indicators": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "World Bank indicator codes, e.g. ['NY.GDP.PCAP.CD', 'MS.MIL.XPND.GD.Z
S']",
                },
                "countries": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "ISO 3166-1 alpha-2 country codes, e.g. ['US', 'CN', 'IR']",
                },
            },
            "required": ["indicators", "countries"],
        },
    },
    {
        "name": "search_news",
        "description": "Search recent news for current events relevant to the scenario.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query":     {"type": "string"},
                "days_back": {"type": "integer", "default": 90, "maximum": 365},
            },
            "required": ["query"],
        },
    },
    {
        "name": "update_simspec",
        "description": (
            "Apply a patch to the current SimSpec. "
            "Use after each user answer or research insight to keep the spec current."
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "patch": {
                    "type": "object",
                    "description": "Partial SimSpec dict. Top-level keys only (actors, theories, timeframe, e
tc.).",
                },
            },
            "required": ["patch"],
        },
    },
    {
        "name": "identify_gap",
        "description": "Register a gap in the SimSpec that needs to be filled.",
```

```
        "input_schema": {
            "type": "object",
            "properties": {
                "field_path":  {"type": "string"},
                "description": {"type": "string"},
                "priority":    {"type": "number", "minimum": 0.0, "maximum": 1.0},
            },
            "required": ["field_path", "description", "priority"],
        },
    },
    {
        "name": "ask_user",
        "description": (
            "Ask the consultant a question to fill a gap. "
            "Returns control to the user — include context and options where helpful."
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "question": {"type": "string"},
                "context":  {"type": "string", "description": "Why you are asking, what research says"},
                "options":  {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "Optional suggested answers",
                },
            },
            "required": ["question"],
        },
    },
    {
        "name": "select_theories",
        "description": "Invoke TheoryMapper to select and parameterize theories based on SimSpec domain and r
esearch context.",
        "input_schema": {"type": "object", "properties": {}, "required": []},
    },
    {
        "name": "finalize",
        "description": "Validate the complete SimSpec and transition to COMPLETE. Only call when all critical
gaps are filled.",
        "input_schema": {"type": "object", "properties": {}, "required": []},
    },
]
```

Tool return types (Python-side)

```
# search_arxiv, search_ssrn  → list[ResearchResult]
# search_fred                → list[DataPoint]
# search_world_bank          → list[DataPoint]
# search_news                → list[NewsItem]
# update_simspec             → SimSpec (updated spec, serialized)
# identify_gap               → SpecGap
# ask_user                   → sentinel: ScopingAgent returns to caller, awaits next turn
# select_theories            → list[TheoryRef]
# finalize                   → {"valid": bool, "errors": list[str]}
```

---

4. Research Pipeline — `forge/researchers/`

All adapters share a common protocol:

```python
from abc import ABC, abstractmethod
from forge.session import ResearchResult, DataPoint, NewsItem


class BaseResearcher(ABC):
    @abstractmethod
    async def search(self, query: str, max_results: int = 5) -> list[ResearchResult]: ...


class BaseDataFetcher(ABC):
    @abstractmethod
    async def fetch(self, *args, **kwargs) -> list[DataPoint]: ...
```

forge/researchers/arxiv.py

```python
import httpx
from forge.session import ResearchResult


class ArxivResearcher(BaseResearcher):
    BASE_URL = "http://export.arxiv.org/api/query"

    async def search(self, query: str, max_results: int = 5) -> list[ResearchResult]:
        """
        HTTP GET to arXiv Atom feed API. Parse XML.
        theory_suggestions and parameter_hints are empty here —
        filled later by ScopingAgent._extraction_pass().
        """
        async with httpx.AsyncClient(timeout=15.0) as client:
            resp = await client.get(self.BASE_URL, params={
                "search_query": f"all:{query}",
                "max_results":  max_results,
                "sortBy":       "relevance",
            })
            resp.raise_for_status()
        return _parse_atom_feed(resp.text)
```

forge/researchers/ssrn.py

```python
import httpx
from forge.session import ResearchResult


class SsrnResearcher(BaseResearcher):
    SEARCH_URL = "https://api.ssrn.com/content/search/paper"

    async def search(self, query: str, max_results: int = 5) -> list[ResearchResult]:
        """
        Queries SSRN search API.
        Falls back to scraping abstract pages if API unavailable.
        Returns list[ResearchResult] with empty theory_suggestions.
        """
        ...
```

forge/researchers/fred.py

```python
import os
import httpx
from forge.session import DataPoint


class FredResearcher(BaseDataFetcher):
    BASE_URL = "https://api.stlouisfed.org/fred/series/observations"

    async def fetch(self, series_ids: list[str]) -> list[DataPoint]:
        """
        One HTTP request per series_id. Fetches most recent 5 observations.
        FRED_API_KEY read from environment.
        """
        api_key = os.environ["FRED_API_KEY"]
        results: list[DataPoint] = []
        async with httpx.AsyncClient(timeout=15.0) as client:
            for sid in series_ids:
                resp = await client.get(self.BASE_URL, params={
                    "series_id":   sid,
                    "api_key":     api_key,
                    "file_type":   "json",
                    "sort_order":  "desc",
                    "limit":       5,
                    "observation_start": "2020-01-01",
                })
                if resp.status_code == 200:
                    results.extend(_parse_fred_response(resp.json(), sid))
        return results
```

forge/researchers/world_bank.py

```python
import httpx
from forge.session import DataPoint


class WorldBankResearcher(BaseDataFetcher):
    BASE_URL = "https://api.worldbank.org/v2/country/{country}/indicator/{indicator}"

    async def fetch(self, indicators: list[str], countries: list[str]) -> list[DataPoint]:
        """
        World Bank v2 API. Fetches most recent value per (country, indicator) pair.
        No API key required.
        """
        ...
```

forge/researchers/news.py

```python
import os
import httpx
from forge.session import NewsItem


class NewsResearcher:
    NEWSAPI_URL = "https://newsapi.org/v2/everything"

    async def search(self, query: str, days_back: int = 90) -> list[NewsItem]:
        """
        Fetch recent news articles via NewsAPI.org.
        NEWS_API_KEY read from environment.
```

```
        """
        ...
```

Extraction pass (consolidation)

After all adapters return, `ScopingAgent._extraction_pass()` calls Claude API once with all summaries:

```
System prompt:
    You are extracting structured information from research summaries.
    For each result, identify:
    1. Which simulation theories it suggests (from registered theory list)
    2. Which SimSpec parameters it calibrates, with numeric estimates

    Respond with JSON:
    {"results": [{"id": "...", "theory_suggestions": [...], "parameter_hints": {...}}]}

User message:
    Domain: {session.domain}
    Registered theories: {list_theories()}
    Research results: [{id, source, title, summary} for each result]
```

The response populates `ResearchResult.theory_suggestions`, `ResearchResult.parameter_hints`, and is consolidated into `ResearchContext.theory_candidates` and `ResearchContext.parameter_estimates`.

---

5. `forge/spec_builder.py` — SpecBuilder

Manages incremental assembly of SimSpec from three input sources.

```python
from __future__ import annotations

from anthropic import Anthropic
from core.spec import SimSpec, ActorSpec, TimeframeSpec
from forge.session import ResearchContext, SpecGap


class SpecBuilder:
    """
    Assembles SimSpec incrementally from:
    1. parse_intake()         — free text → skeleton SimSpec
    2. apply_research_hints() — ResearchContext → initial_environment
    3. apply_user_answer()    — user message → targeted SimSpec patch
    4. infer_gap()            — fills remaining gaps from inference
    """

    def __init__(self, client: Anthropic | None = None) -> None:
        self.client = client or Anthropic()

    def parse_intake(self, text: str) -> SimSpec:
        """
        Claude API call with structured output prompt.
        Extracts: name, description, domain, actors (names only), rough timeframe.
        Returns minimal SimSpec — actors have names but empty beliefs/desires.

        Prompt: "Parse this scenario. Return JSON matching SimSpec schema.
                 Focus on: name, description, domain, actors (name + description only),
                 timeframe (total_ticks, tick_unit),
```

```python
                initial_environment keys (set to 0.5 as placeholder)."
        """
        ...

    def apply_research_hints(self, spec: SimSpec, ctx: ResearchContext) -> None:
        """
        Merge ctx.parameter_estimates into spec.initial_environment.
        Marks env_keys_calibrated in ctx.
        Updates spec in-place.
        """
        for key, value in ctx.parameter_estimates.items():
            spec.initial_environment[key] = value
            ctx.env_keys_calibrated.add(key)

    def apply_user_answer(
        self,
        spec: SimSpec,
        gaps: list[SpecGap],
        user_message: str,
    ) -> None:
        """
        Claude API call with:
        - current spec (abridged)
        - the gap question that was just asked
        - user_message

        Extracts structured patch. Applies to spec in-place.
        Marks relevant gaps as filled.
        """
        ...

    def infer_gap(
        self,
        spec: SimSpec,
        gap: SpecGap,
        ctx: ResearchContext,
    ) -> None:
        """
        Fill gap.field_path with a defensible default:
        - If ctx.parameter_estimates has a relevant key, use it.
        - Otherwise apply domain-appropriate defaults from DEFAULT_ENV_VALUES.
        - Record inference in spec.metadata["inferred_fields"].
        """
        ...

    def patch_spec(self, spec: SimSpec, patch: dict) -> SimSpec:
        """
        Merge patch dict into spec. Returns new SimSpec instance (immutable).
        Uses model_copy(update=patch) to avoid mutation during WS reads.
        """
        return spec.model_copy(update=patch)


# Domain defaults — applied when research and user provide no value
DEFAULT_ENV_VALUES: dict[str, dict[str, float]] = {
    "geopolitics": {
        "global__stability":            0.5,
        "global__escalation_risk":      0.3,
        "global__diplomatic_activity":  0.5,
    },
    "market": {
        "global__market_concentration": 0.5,
        "global__competitive_intensity": 0.5,
        "global__demand_growth":         0.3,
    },
    "macro": {
        "global__gdp_growth":    0.02,
        "global__inflation":     0.03,
        "global__unemployment":  0.05,
    },
```

```
    "org": {
        "global__adoption_rate":    0.1,
        "global__resistance":       0.4,
        "global__leadership_buy_in": 0.6,
    },
}
```

---

6. Gap Detection — `forge/gap_detector.py`

Standalone, zero-dependency module. Implement and test this first.

```
from core.spec import SimSpec
from forge.session import SpecGap


# Priority constants
P_ACTORS      = 1.0
P_BELIEFS     = 0.9
P_THEORIES    = 0.85
P_ENVIRONMENT = 0.75
P_TIMEFRAME   = 0.6
P_METRICS     = 0.5
P_UNCERTAINTY = 0.3


def detect_gaps(spec: SimSpec) -> list[SpecGap]:
    """
    Inspect spec for missing or under-specified fields.
    Returns list[SpecGap] ordered by priority descending.

    Rule table:
    ┌──────────────────────────────────────────────┬──────────┐
    │ Condition                                    │ Priority │
    ├──────────────────────────────────────────────┼──────────┤
    │ spec.actors is empty                         │ 1.0      │
    │ any actor has no beliefs                     │ 0.9      │
    │ any actor has no desires                     │ 0.85     │
    │ spec.theories is empty                       │ 0.85     │
    │ spec.initial_environment is empty            │ 0.75     │
    │ no env keys for any actor                    │ 0.75     │
    │ spec.timeframe.total_ticks == 365 (default)  │ 0.6      │
    │ spec.metrics is empty                        │ 0.5      │
    │ spec.uncertainty still at all-defaults       │ 0.3      │
    └──────────────────────────────────────────────┴──────────┘
    """
    gaps: list[SpecGap] = []

    if not spec.actors:
        gaps.append(SpecGap(
            field_path="actors",
            description="No actors identified. Who are the key decision-makers?",
            priority=P_ACTORS,
        ))
        return gaps  # no point checking sub-fields if no actors

    for actor in spec.actors:
        if not actor.beliefs:
            gaps.append(SpecGap(
                field_path=f"actors[{actor.name}].beliefs",
                description=f"Actor '{actor.name}' has no beliefs defined.",
                priority=P_BELIEFS,
            ))
```

```python
        if not actor.desires:
            gaps.append(SpecGap(
                field_path=f"actors[{actor.name}].desires",
                description=f"Actor '{actor.name}' has no objectives defined.",
                priority=P_BELIEFS,
            ))

    if not spec.theories:
        gaps.append(SpecGap(
            field_path="theories",
            description="No simulation theories selected.",
            priority=P_THEORIES,
        ))

    if not spec.initial_environment:
        gaps.append(SpecGap(
            field_path="initial_environment",
            description="No environment variables seeded.",
            priority=P_ENVIRONMENT,
        ))
    else:
        all_env_keys = set(spec.initial_environment.keys())
        for actor in spec.actors:
            actor_prefix = f"{actor.name.lower().replace(' ', '_')}__"
            has_keys = any(k.startswith(actor_prefix) for k in all_env_keys)
            if not has_keys and not actor.initial_env_contributions:
                gaps.append(SpecGap(
                    field_path=f"actors[{actor.name}].initial_env_contributions",
                    description=f"No environment variables owned by actor '{actor.name}'.",
                    priority=P_ENVIRONMENT,
                ))

    if spec.timeframe.total_ticks == 365 and spec.timeframe.tick_unit == "day":
        gaps.append(SpecGap(
            field_path="timeframe",
            description="Timeframe is at default (365 days). Is this the right horizon?",
            priority=P_TIMEFRAME,
        ))

    if not spec.metrics:
        gaps.append(SpecGap(
            field_path="metrics",
            description="No outcome metrics defined. What does success or failure look like?",
            priority=P_METRICS,
        ))

    return sorted(gaps, key=lambda g: g.priority, reverse=True)


def _merge_gaps(existing: list[SpecGap], new_gaps: list[SpecGap]) -> None:
    """
    Merge new_gaps into existing in-place.
    Preserves filled status of existing gaps.
    Adds new gaps not already in existing.
    """
    existing_paths = {g.field_path for g in existing}
    for gap in new_gaps:
        if gap.field_path not in existing_paths:
            existing.append(gap)
```

---

7. Theory Mapper — `forge/theory_mapper.py`

Deterministic component — no LLM calls. Maps domain + research to a prioritized `list[TheoryRef]`.

```python
from __future__ import annotations

from core.spec import SimSpec, TheoryRef
from core.theories import list_theories, get_theory
from forge.session import ResearchContext


# Domain → candidate theories
DOMAIN_THEORY_MAP: dict[str, list[str]] = {
    "geopolitics": [
        "richardson_arms_race",   # escalation dynamics
        "wittman_zartman",        # negotiated settlement
        "fearon_bargaining",      # war onset / termination
    ],
    "market": [
        "porter_five_forces",
        "supply_demand_shock",
        "market_contagion",
    ],
    "macro": [
        "keynesian_multiplier",
        "regulatory_shock",
        "trade_flow_gravity",
    ],
    "org": [
        "principal_agent",
        "diffusion_of_innovation",
        "institutional_inertia",
    ],
}

# Theories that cannot compose — pick the higher-scoring one
THEORY_CONFLICTS: list[tuple[str, str]] = [
    ("wittman_zartman", "fearon_bargaining"),  # competing termination models
]

# Priority within SimRunner tick loop (lower = runs first)
THEORY_PRIORITY: dict[str, int] = {
    "richardson_arms_race":  0,
    "fearon_bargaining":     1,
    "wittman_zartman":       1,
    "keynesian_multiplier":  0,
    "porter_five_forces":    0,
    "supply_demand_shock":   1,
    "market_contagion":      2,
}


class TheoryMapper:
    """
    Algorithm:
    1. Start with domain candidates from DOMAIN_THEORY_MAP.
    2. Score: base 0.5 + research_boost (normalized research mentions).
    3. Filter to registered theories only.
    4. Conflict resolution: keep higher-scoring of conflicting pair.
    5. Select top MAX_THEORIES.
    6. Initialize parameters from ResearchContext.parameter_estimates.
    7. Assign priorities from THEORY_PRIORITY.
    """

    MAX_THEORIES = 3

    def map(self, spec: SimSpec, ctx: ResearchContext) -> list[TheoryRef]:
        domain = spec.domain or "geopolitics"
        candidates = DOMAIN_THEORY_MAP.get(domain, list(list_theories()))

        # Filter to registered theories
        registered = set(list_theories())
```

```python
        candidates = [t for t in candidates if t in registered]

        scores = self._score_candidates(candidates, ctx)
        candidates = self._resolve_conflicts(candidates, scores)
        selected = sorted(candidates, key=lambda t: scores[t], reverse=True)[: self.MAX_THEORIES]

        refs: list[TheoryRef] = []
        for theory_id in selected:
            params = self._init_parameters(theory_id, ctx)
            refs.append(TheoryRef(
                theory_id=theory_id,
                priority=THEORY_PRIORITY.get(theory_id, 99),
                parameters=params,
            ))

        return sorted(refs, key=lambda r: r.priority)

    def _score_candidates(self, candidates: list[str], ctx: ResearchContext) -> dict[str, float]:
        scores: dict[str, float] = {t: 0.5 for t in candidates}
        total = len(ctx.results) or 1
        for result in ctx.results:
            for suggestion in result.theory_suggestions:
                if suggestion in scores:
                    scores[suggestion] += result.relevance_score / total
        return scores

    def _resolve_conflicts(self, candidates: list[str], scores: dict[str, float]) -> list[str]:
        excluded: set[str] = set()
        for t1, t2 in THEORY_CONFLICTS:
            if t1 in candidates and t2 in candidates:
                loser = t1 if scores.get(t1, 0) < scores.get(t2, 0) else t2
                excluded.add(loser)
        return [t for t in candidates if t not in excluded]

    def _init_parameters(self, theory_id: str, ctx: ResearchContext) -> dict[str, float]:
        """
        Pull parameter values from ctx.parameter_estimates where keys match
        "{theory_id}__{param_name}". Fall back to TheoryBase.Parameters defaults.
        """
        prefix = f"{theory_id}__"
        params: dict[str, float] = {}
        for key, value in ctx.parameter_estimates.items():
            if key.startswith(prefix):
                params[key[len(prefix):]] = value
        return params
```

---

8. Dynamic Question Generation

Questions generated by Claude API inside `ScopingAgent._generate_question()`. Research context is baked in so every question is grounded.

```
QUESTION_GENERATION_PROMPT = """
You are helping a consultant configure a simulation scenario.
Ask ONE question to fill the following gap in the simulation spec.

Gap: {gap.description}
Field: {gap.field_path}
Scenario so far: {spec_summary}
Research findings: {research_summary}

Rules:
- Ask in consulting language. No model parameters, no jargon.
```

```
- If research partially answers this, acknowledge it and ask for confirmation.
- If offering options, limit to 3-4 concrete choices.
- 2-3 sentences maximum.
- Do not reveal that you are building a SimSpec.

Examples of good questions:
  "Based on recent IISS data, Iran's military spending is approximately 2.1% of GDP.
   Does that align with your assessment, or do you see it meaningfully higher or lower?"

  "Who are the primary decision-makers in this scenario?
   For example: the acquiring firm's board, the target's management, and key regulators?
   Or are there others you'd add?"

Write the question:
"""


RESEARCH_SUMMARY_TEMPLATE = """
Key research findings: {findings}
Already calibrated from research: {calibrated_keys}
Parameter estimates available: {param_count} values
Theory candidates identified: {theory_candidates}
"""
```

Question budget rule: The agent asks at most `MAX_TURNS = 5` questions. Remaining unfilled gaps after turn 5 are resolved by `_auto_fill_gaps()` using inference + domain defaults. The consultant is notified: "*I've made some reasonable assumptions for the remaining parameters — you can review them in the spec before running.*"

---

9. Full Flow Diagram

```
Consultant: "Model the impact of EU carbon tariffs on German auto manufacturers..."
        │
        ▼
POST /forge/sessions  ──►  ForgeSession(state=INTAKE)
        │
        ▼
ScopingAgent._run_intake()
  │
  ├─ parse_intake(text) via Claude API
  │    → SimSpec(name="EU Carbon Tariff Impact", domain="market",
  │              actors=[ActorSpec("German OEM"), ActorSpec("EU Commission")],
  │              timeframe=TimeframeSpec(total_ticks=365))
  │
  ├─ state → PARALLEL_RESEARCH
  │
  ├─ asyncio.gather(
  │    ArxivResearcher.search("carbon border adjustment mechanism auto industry"),
  │    SsrnResearcher.search("EU ETS automotive competitiveness"),
  │    FredResearcher.fetch(["GEPUCURRENT", "DEUPROINDMISMEI"]),
  │    WorldBankResearcher.fetch(["EN.ATM.CO2E.PC"], ["DE", "FR", "PL"]),
  │    NewsResearcher.search("EU carbon tariff auto 2026", days_back=90),
  │  )  → 5 ResearchResult, 6 DataPoint, 4 NewsItem
  │
  ├─ _extraction_pass(ctx) via Claude API
  │    → theory_candidates: ["porter_five_forces", "supply_demand_shock"]
```

```
  │    → parameter_estimates: {"porter_five_forces__rivalry": 0.72,
  │                            "global__carbon_price": 0.65}
  │
  ├─ apply_research_hints(spec, ctx)
  │    → spec.initial_environment updated
  │
  ├─ detect_gaps(spec)
  │    → [SpecGap("actors[German OEM].beliefs", priority=0.9),
  │        SpecGap("actors[EU Commission].desires", priority=0.85),
  │        SpecGap("metrics", priority=0.5)]
  │
  ├─ _mark_research_filled_gaps()
  │
  └─ state → DYNAMIC_INTERVIEW
        │
        ▼
_generate_question(gaps[0]) via Claude API
  → "Arora et al. (2024) found EU OEMs face roughly a 4-7% cost increase
     under CBAM. How do you expect German manufacturers to respond —
     primarily through lobbying, technology investment, or production shifts?"
        │
        ▼
POST /forge/sessions/{id}/messages
  user: "Primarily technology investment, but lobbying will be significant in year 1"
        │
        ▼
apply_user_answer() via Claude API
  → spec.actors[0].beliefs, desires updated
  → gap marked filled
        │
        ▼
[turns 2–5: similar, closing remaining gaps]
        │
        ▼
turn_count >= MAX_TURNS  OR  open_gaps() == []
        │
        ▼
TheoryMapper.map(spec, ctx)
  → [TheoryRef("porter_five_forces", priority=0, parameters={rivalry: 0.72}),
     TheoryRef("supply_demand_shock", priority=1, parameters={shock_size: 0.15})]
        │
        ▼
SimSpec.model_validate()  ── passes ──►  state = COMPLETE
        │
        ▼
SimRunner(spec).setup() → run_async()
```

---

10. API Integration — `api/routers/forge.py`

```python
from __future__ import annotations

import asyncio
from fastapi import APIRouter, WebSocket, WebSocketDisconnect, HTTPException
from pydantic import BaseModel

from forge.scoping_agent import ScopingAgent
from forge.session import ForgeSession, ForgeState
from core.spec import SimSpec
from core.sim_runner import SimRunner


router = APIRouter(prefix="/forge", tags=["forge"])

# In-memory session store (replace with Redis/DB for production)
_sessions: dict[str, ForgeSession] = {}


# ── Request / response models ──────────────────────────────────────

class CreateSessionRequest(BaseModel):
    intake_text: str

class CreateSessionResponse(BaseModel):
    session_id: str
    state: str

class SendMessageRequest(BaseModel):
    message: str

class SendMessageResponse(BaseModel):
    session_id:     str
    state:          str
    reply:          str
    turn_count:     int
    open_gap_count: int

class ForgeSessionView(BaseModel):
    session_id:   str
    state:        str
    turn_count:   int
    domain:       str
    gap_count:    int
    open_gaps:    int
    created_at:   float
    completed_at: float | None


# ── Endpoints ──────────────────────────────────────────────────────

@router.post("/sessions", response_model=CreateSessionResponse)
async def create_session(body: CreateSessionRequest) -> CreateSessionResponse:
    """
    Create a new ForgeSession. Immediately starts research in the background.
    Client connects to WS /sessions/{id}/stream for live research progress,
    then polls GET /sessions/{id} until state == "dynamic_interview".
    """
    agent = ScopingAgent()
    session = ScopingAgent.create_session(body.intake_text)
    _sessions[session.session_id] = session
    asyncio.create_task(_run_first_turn(agent, session))
    return CreateSessionResponse(session_id=session.session_id, state=session.state.value)


async def _run_first_turn(agent: ScopingAgent, session: ForgeSession) -> None:
    try:
        await agent.turn(session, user_message=None)
    except Exception:
```

```python
        session.state = ForgeState.INTAKE  # allow retry


@router.get("/sessions/{session_id}", response_model=ForgeSessionView)
async def get_session(session_id: str) -> ForgeSessionView:
    session = _get_session(session_id)
    return ForgeSessionView(
        session_id=session.session_id,
        state=session.state.value,
        turn_count=session.turn_count,
        domain=session.domain,
        gap_count=len(session.gaps),
        open_gaps=len(session.open_gaps()),
        created_at=session.created_at,
        completed_at=session.completed_at,
    )


@router.post("/sessions/{session_id}/messages", response_model=SendMessageResponse)
async def send_message(session_id: str, body: SendMessageRequest) -> SendMessageResponse:
    session = _get_session(session_id)
    if session.state == ForgeState.COMPLETE:
        raise HTTPException(400, "Session already complete.")
    agent = ScopingAgent()
    reply = await agent.turn(session, user_message=body.message)
    return SendMessageResponse(
        session_id=session_id,
        state=session.state.value,
        reply=reply,
        turn_count=session.turn_count,
        open_gap_count=len(session.open_gaps()),
    )


@router.post("/sessions/{session_id}/finalize")
async def finalize_session(session_id: str) -> dict:
    """Manually trigger finalization — useful if consultant wants to proceed early."""
    session = _get_session(session_id)
    agent = ScopingAgent()
    session.state = ForgeState.THEORY_MAPPING
    reply = await agent.turn(session)
    return {"state": session.state.value, "message": reply}


@router.get("/sessions/{session_id}/simspec")
async def get_simspec(session_id: str) -> dict:
    """Return current SimSpec as JSON. Live — updates as session progresses."""
    session = _get_session(session_id)
    if session.simspec is None:
        raise HTTPException(404, "SimSpec not yet initialized.")
    return session.simspec.model_dump()


@router.websocket("/sessions/{session_id}/stream")
async def stream_session(websocket: WebSocket, session_id: str) -> None:
    """
    WebSocket for live progress during PARALLEL_RESEARCH.

    Events emitted:
      {"event": "research_result", "source": "arxiv", "title": "...", "relevance_score": 0.8}
      {"event": "gap_detected",    "field_path": "actors", "priority": 1.0}
      {"event": "state_change",    "from": "parallel_research", "to": "dynamic_interview"}
      {"event": "question",        "text": "..."}
      {"event": "complete",        "spec_id": "...", "actor_count": 2, "theory_count": 2}

    Stream closes when state = COMPLETE.
    Researchers push events to an asyncio.Queue on the session;
    this handler drains the queue at 200ms intervals.
    """
    await websocket.accept()
```

```
    session = _sessions.get(session_id)
    if not session:
        await websocket.close(code=4004)
        return
    try:
        while session.state != ForgeState.COMPLETE:
            await asyncio.sleep(0.2)
            # drain session.event_queue → send JSON over WS
    except WebSocketDisconnect:
        pass


def _get_session(session_id: str) -> ForgeSession:
    if session_id not in _sessions:
        raise HTTPException(404, f"Session {session_id!r} not found.")
    return _sessions[session_id]
```

---

11. Key Design Decisions

Decision	Rationale
Claude API tool-use vs. custom orchestration	Tool-use lets Claude decide research sequencing based on what each result reveals. A hand-coded orchestration loop would need to anticipate every possible research path across every domain.
Parallel research before first question	Research takes 3–8 seconds per source. Running it before any question means the consultant receives a research-informed first question within ~15 seconds, rather than a generic "who are the actors?" immediately. It also eliminates questions research can already answer.
Gap-driven question generation vs. fixed set	Fixed question sets break on scenario variety — geopolitics and pharma M&A have nothing in common. Gap detection reads actual SimSpec state and generates targeted questions. `MAX_TURNS = 5` is a hard budget, not a template length.
TheoryMapper as a separate component	Mapping logic (domain classification, scoring, conflict detection, parameter init) is complex enough to test independently and has no LLM dependency — fully deterministic.
`MAX_TURNS = 5`	User research on agentic intake shows abandonment above 5 questions. The spec can be refined after first run, so it is better to start sooner with reasonable defaults.
Research failure is non-fatal (`return_exceptions=True`)	External APIs (SSRN, news) are unreliable. A single adapter failure must not block the session. Proceed with whatever returned; log which sources were unavailable in `ResearchContext.metadata`.
`model_copy(update=...)` not in-place mutation on SimSpec	SimSpec is Pydantic v2. Mutating it while the WS stream reads it creates race conditions. `patch_spec()` uses `model_copy()` and atomically assigns the new instance to `session.simspec`.
Contradictory user answers	`apply_user_answer()` takes the most recent answer as authoritative. Previous conflicting values are overwritten; logged in `spec.metadata["overwritten_values"]`.

`ResearchSourceSpec` on SimSpec	The final SimSpec records which sources calibrated it. This creates an audit trail and lets the Data Feed Agent re-query the same sources for recalibration as real-world data evolves.

---

12. Module Layout

```
forge/
├── __init__.py
├── scoping_agent.py       ← ScopingAgent, state machine, tool dispatch
├── session.py             ← ForgeSession, ForgeState, ForgeMessage, SpecGap,
│                             ResearchResult, DataPoint, NewsItem, ResearchContext
├── gap_detector.py        ← detect_gaps(), _merge_gaps(), priority constants
├── theory_mapper.py       ← TheoryMapper, DOMAIN_THEORY_MAP, THEORY_CONFLICTS
├── spec_builder.py        ← SpecBuilder, parse_intake, apply_*, infer_gap, DEFAULT_ENV_VALUES
└── researchers/
    ├── __init__.py        ← BaseResearcher, BaseDataFetcher
    ├── arxiv.py           ← ArxivResearcher
    ├── ssrn.py            ← SsrnResearcher
    ├── fred.py            ← FredResearcher
    ├── world_bank.py      ← WorldBankResearcher
    └── news.py            ← NewsResearcher

api/
└── routers/
    └── forge.py           ← FastAPI router: /forge/sessions, WS /sessions/{id}/stream
```

Implementation order:

1. `forge/session.py` — all data types (everything imports from here)
2. `forge/gap_detector.py` — standalone, zero deps, testable in isolation
3. `forge/theory_mapper.py` — deterministic, no LLM, testable in isolation
4. `forge/researchers/` — one adapter at a time (arxiv first — no API key)
5. `forge/spec_builder.py` — Claude API calls
6. `forge/scoping_agent.py` — full state machine, requires all above
7. `api/routers/forge.py` — wires everything to FastAPI

---

13. Environment Variables Required

```
ANTHROPIC_API_KEY   # Claude API — ScopingAgent, extraction_pass, parse_intake, generate_question
FRED_API_KEY        # FRED data fetcher
NEWS_API_KEY        # NewsAPI.org
# World Bank: no API key required
# arXiv: no API key required
# SSRN: no API key required (rate-limited)
```

Next: ARCHITECTURE-API.md

The Forge hands a complete `SimSpec` to `SimRunner`. The next architecture document covers:

- `POST /simulations` — accept SimSpec, instantiate and start SimRunner
- `GET /simulations/{id}/snapshots` — list and retrieve SimSnapshot objects
- `GET /simulations/{id}/metrics` — time series of MetricRecord
- `POST /simulations/{id}/shocks` — inject scheduled_shocks at runtime
- `GET /simulations/{id}/stream` — WebSocket for live tick-by-tick env updates
- Session persistence: ForgeSession and SimRunner state in Redis or SQLite
- Authentication: API keys for Portal (client-facing) vs. internal Forge access
- The Data Feed Agent: cron-driven recalibration via ResearchSourceSpec